

Staff Desk Workspaces, Better Auth & Member QR System

Detailed functional walkthrough of every Staff Desk feature from the navigation sidebar, how Better Auth operates under the hood, and how the cryptographic member QR cards are issued, rendered, and verified.

Monolith: Next.js 16 · React 19 · Prisma 7 · PostgreSQL **Auth Engine:** Better Auth (v1.7.7) **QR Suite:** qrcode + @zxing/browser

Focus: Real-world Operations & Workflows

1. What is Better Auth & Why It's Used Here

Better Auth is a TypeScript-first, self-hosted authentication library for Next.js and Node.js. Unlike third-party cloud auth providers (Clerk, Auth0, Firebase), Better Auth runs **100% inside this application** and writes directly to our local PostgreSQL database using Prisma.

❌ Cloud SaaS Auth (Clerk / Auth0)

Stores user records on external servers, requires monthly paid fees, adds 200ms+ network roundtrips, breaks relational joins with bookings/orders, and cannot run offline.

✅ Better Auth (Self-Hosted in Champions Club)

Zero monthly fees. All user accounts, sessions, password hashes, and rate-limits live directly inside PostgreSQL tables. Works offline with zero third-party dependencies.

The 5 Database Tables Better Auth Controls

1. User

Stores identity: name, email, role (MEMBER, CASHIER, etc.), Champions ID (CC-XXXX), and emailVerified status.

2. Session

Active browser sessions (token, userId, expiresAt, ipAddress) linked to an `httpOnly` cookie.

3. Account

Stores password hashes (argon2/scrypt). Kept isolated from profile information.

4. Verification

Transient 1-hour cryptographic tokens for email verification links and password resets.

5. RateLimit

Database-backed sliding window counters (5 signups/min, 10 signins/min) stopping brute-force attacks without Redis.

2. Authentication & The "Dual Guard" Rule

When a user registers or logs in, how does the system prevent privilege escalation and enforce immediate role changes?

🔒 Security Invariant: Zero Privilege Escalation at Signup

In `src/lib/auth.ts`, both `role` and `championsId` have `input: false`. A transactional database hook forces every public signup to `role = "MEMBER"` and generates a random Champions ID (e.g. `CC-4A92F1`). Even if an attacker injects `{ "role": "OWNER" }` into the HTTP payload, it is rejected by Better Auth.

The Dual Guard (`src/lib/access.ts`): Most apps trust a session cookie or JWT for 7 days. But what if the Owner demotes an employee from `RECEPTION` to `MEMBER`? A cached cookie would let them keep snooping on bookings! Champions Club uses a 2-step verification on every request:

1. Better Auth validates the cryptographic session cookie token.
2. **Immediate Database Lookup:** The server re-reads the fresh `User` row directly from PostgreSQL. If the role was changed or revoked, access is denied **instantaneously** on that exact HTTP request!

3. Deep Dive into the Staff Workspace (Your Screenshot)

The screenshot you shared shows the **Owner Navigation Sidebar** in `src/components/staff-desk.tsx`. When a user with the **OWNER** role logs in and visits `/staff`, they have complete operational control over these 6 core functions:

Club desk

Member lookup

Business settings

Membership plans

Staff access

Local test inbox

Staff Sidebar UI

Menu Items in Screenshot:

- Club desk:** The main operations dashboard showing live counters and actionable tasks.
- [] Member lookup:** Identity station for scanning QR cards or searching by Champions ID / email.
- ⚙️ Business settings:** Owner configuration for operating hours, booking windows, and tab limits.
- 👤 Membership plans:** Plan designer for prices (in paise), terms, and discount basis points.
- 👤 Staff access:** Role management panel for promoting staff and revoking active sessions.
- ✉️ Local test inbox:** In-browser email viewer for offline verification links & password resets.

1. Club Desk (Tab: `id: "today"`) — Active Tab

ALL STAFF & OWNER

What it is: The front-line operational cockpit that staff see the moment they open the app.

What it does on screen:

- Live Action Badges:** Queries `GET /api/staff/home` every 5 seconds to display large metric cards: "Today's Court Bookings: 8", "Unread CRM Enquiries: 3", "Low Stock Inventory Items: 2", "Open Kitchen Tickets: 4", and "Active Cash Shift: OPEN".
- One-Click Workflow Navigation:** Clicking any metric card immediately switches the view to that specific workspace. For example, clicking "3 Unread Enquiries" instantly opens the CRM desk so reception can respond to the customer.
- Role Banner:** Displays the logged-in staff member's name and role badge (e.g. "Hello, Arjun. Reception workspace").

2. [] Member Lookup (Tab: `id: "lookup"`)

RECEPTION, CASHIERS, OWNER

What it is: The customer identification station used at Reception, Pro Shop, and Clubhouse dining.

Real-World Workflow:

- Scenario:** A customer walks up to the tennis desk to check in or arrives at the clubhouse restaurant table.
- Camera QR Scan:** Staff clicks "Scan QR camera". The device webcam activates. The customer holds up their phone displaying their digital QR card from `/account`. The camera decodes the QR token in under 200 milliseconds.
- Manual Fallback:** If the customer doesn't have their phone, staff can type their **Champions ID** (e.g., `CC-4A92F1`) or registered **email address**.
- Output Shown to Staff:** The system displays the member's full name, Champions ID, active membership tier (e.g. "Gold Plan · 25% court, 15% shop, 10% clubhouse discount"), and the plan expiration date.
- Security Rule:** It verifies identity and eligibility; it **never automatically debits money** or exposes passwords.

3. ⚙️ Business Settings (Tab: `id: "settings"`)

OWNER EXCLUSIVE

What it is: The owner's policy control room where club operating rules are established and stored in the `ClubSettings` table.

Key Rules Controlled by the Owner:

- Club Hours:** Opening (e.g. `06:00`) and Closing hour (e.g. `23:00` Asia/Kolkata). Courts cannot be booked outside these hours.
- Booking Window (Days):** (Default: `14` days). Members cannot reserve slots months in advance, preventing slot hoarding.
- Daily Session Limit:** (Default: `2` sessions). Prevents a single player from booking 5 hours of courts in a single day.
- Checkout Hold Duration:** (Default: `5` minutes). When a player clicks "Book", the court is locked for 5 minutes. If they abandon checkout or close their laptop, the database worker automatically releases the slot for other members.
- Cancellation Notice Hours:** (Default: `12` hours). Customers must cancel at least 12 hours before game time to receive a refund/credit. Cancellations inside 12 hours require staff override with a recorded reason.
- Member Bar Tab Limit:** (Default: `₹5,000` / 500,000 paise). The maximum credit a member can run up at the restaurant/bar before their account is blocked from further credit.
- Automated Reminder Schedule:** (Default: `7, 1, 0` days). The background worker automatically schedules expiry reminder emails 7 days before, 1 day before, and on the day a membership expires at 09:00 AM.

4. 👤 Membership Plans (Tab: `id: "plans"`)

OWNER EXCLUSIVE

What it is: The product catalog designer where the owner builds and modifies membership packages (Standard, Gold, Junior).

What the Owner Configures:

- **Plan Price in Integer Paise:** Stored as integers to prevent rounding bugs (e.g. ₹12,000 is stored as `1200000`).
- **Term Duration:** Number of days the membership lasts (e.g. `365` days for annual, `30` days for monthly).
- **Discounts in Basis Points:** 100 basis points = 1%. For example, setting `courtDiscountBps = 2500` grants a **25% discount** on all court reservations.
- **Clubhouse & Shop Discounts:** Separate discount percentages for restaurant dining and pro shop merchandise.
- **Free Weekly Court Quotas:** Complimentary court hours granted to premium tiers (e.g. Gold members receive 1 free session per week).
- **Plan Versioning & Immutability:** If the owner raises Gold membership from ₹12,000 to ₹15,000, **existing members are not affected**. Past invoices and purchased memberships retain their historical price snapshot forever. The new price receives a fresh `planVersion` timestamp.

5. 👤 Staff Access (Tab: `id: "users"`)

OWNER EXCLUSIVE

What it is: The owner's employee onboarding and Role-Based Access Control (RBAC) portal.

How an Employee is Granted Staff Access:

1. The employee registers a regular account on `/signup` with their email address.
2. The Owner opens **Staff access**, types the employee's email, and chooses their role from the dropdown:
 - RECEPTION** Court calendar, check-ins, CRM lead quotes, member lookup.
 - CASHIER** Waiter POS tables, shop counter checkout, cash reconciliation.
 - KITCHEN** Kitchen Display System (KDS), cooking queue, marking tickets ready.
 - OWNER** Unrestricted master access to payroll, finance, settings, and reports.
3. **Instant Session Revocation:** The moment the owner clicks *"Assign role"*, the server updates the database AND runs: `await tx.session.deleteMany({ where: { userId: user.id } })`. This terminates all active sessions across all devices for that employee, forcing them to sign in again with their new staff permissions.

6. ✉ Local Test Inbox (Tab: `id: "inbox"`)

OWNER EXCLUSIVE (LOCAL MODE)

What it is: An embedded, in-browser email client built specifically for local development, hackathons, and offline testing.

Why it exists & how to use it:

- Real SMTP providers (SendGrid, AWS SES) require credit cards, domain verification, and an internet connection.
- When running locally (`EMAIL_MODE=local`), the server intercepts all outgoing emails (signup verification, password reset links, membership expiry notices) and writes them to PostgreSQL's `MailMessage` table.
- Staff/Developers simply open this tab to see outgoing emails in real-time.
- Inside the email body is a blue link: **"Open secure local link ↗"**. Clicking it verifies your test account or resets your password immediately with zero external dependencies!

4. How the Member QR Card is Made (Generation)

Every active member receives a digital membership card on their `/account` dashboard (generated in `src/modules/account/cards.ts`).

Step 1: Opaque Cryptographic Token

The server checks that the user has an active membership, then generates a 48-character random hex token:

```
const token = randomBytes(24).toString("hex");
await tx.memberCard.upsert({
  where: { userId },
  create: { userId, token },
  update: { token, revokedAt: null, issuedAt: now }
});
```

Step 2: Base64 PNG Rendering

The payload `champions:card:<token>` is converted into a PNG data URL using the `qrcode` package:

```
const qr = await QRCode.toDataURL(
  `champions:card:${card.token}`,
  { width: 240, margin: 2, errorCorrectionLevel: "M" }
);
```

🔒 The Opaque Token Principle (Zero Personal Data in QR)

The QR matrix **never contains the member's name, email, phone number, balance, or discounts**. If someone takes a photo of the card, they acquire zero credentials. Members can click **"Revoke card"** anytime on `/account` to invalidate the token immediately.

5. How the Member QR Card is Read & Verified

Scanning is handled by staff via `MemberFinder` in `src/components/staff-operations.tsx`.

📹 1. Live Camera Stream (@zxing/browser)

Accesses the webcam via WebRTC. Zebra Crossing decodes video frames continuously:

```
const reader = new BrowserQRCodeReader();
await reader.decodeFromVideoDevice(
  undefined, videoRef, (res, err, controls) => {
    if (res) {
      controls.stop(); // Stop camera once read
      lookupMemberApi(res.getText());
    }
  }
);
```

🔍 2. Database Resolution (/api/staff/lookup)

The server validates the token and confirms that `revokedAt` is null:

```
const token = query.slice(15);
const card = await db.memberCard.findUnique({
  where: { token }, include: { user: true }
});
if (!card || card.revokedAt) throw new AppError(404, "CARD_INVALID");
const membership = await currentMembership(card.userId);
return { name: card.user.name, membership };
```

⚠️ Identification ≠ Payment Authority

In poorly designed club systems, scanning a QR card automatically charges the user or opens an unlimited bar tab. In Champions Club, **a QR scan ONLY identifies who the player is and which membership tier they hold**. Any financial transaction (paying an invoice, recording a cash/UPI payment, court booking) requires a separate, explicit payment authorization flow.

6. Summary Reference Table for Frontend Work

Subsystem	Component File	What You Need to Know When Editing Frontend
Auth Client	<code>src/lib/auth-client.ts</code>	Call <code>authClient.signIn.email()</code> , <code>authClient.signOut()</code> , or <code>authClient.useSession()</code> .
Staff Desk UI	<code>src/components/staff-desk.tsx</code>	Tabs filter dynamically by <code>user.role</code> . Only <code>OWNER</code> sees Settings, Plans, Access, and Inbox.
QR Display	<code>src/components/account.tsx</code>	Renders base64 data URL returned by <code>GET /api/me/card</code> inside the member card view.
QR Camera	<code>src/components/staff-operations.tsx</code>	<code>MemberFinder</code> manages webcam stream, ZXing barcode decoding, and API lookup.